# Software Architecture and Design Coursework

G9-SAD

Jack Turner A00011257
Luke Pring A00012218
Azlan Rashid A00039826
Sameer Shabbir A00016610

https://github.com/lukepring/Stock-Comparison-Java

# Contents


| | |
|---|---|
| Title Page | 1 |
| Contents Page | 2 |
| Architecture Compliance | 3 |
| Business Concept Model | 4 |
| Business Type Model | 5 |
| Component Specification Diagram | 6 - 8 |
| Dependency Structure | 9 |
| Initial System Architecture Diagram | 10 |
| Use Case Model | 11 - 12 |
| Interface Diagrams | 13 |
| Compound Component Diagram | 14 - 15 |
| Architectural Patterns | 16 - 17 |
| Meetings | 18 - 25 |
| Test Cases | 26 - 28 |

# Architecture Compliance

**Overview**

The system has been designed using a layered structure based on Clean Architecture principles. This approach was chosen to keep the application organised and to make it easier to maintain and extend as new features are added in later stages of development.

**Layered Structure**

The architecture is divided into four main layers: presentation, application, domain, and infrastructure. The presentation layer is responsible for handling user interaction through the Web UI and passing requests to the controller. The application layer manages the main flow of the system, including retrieving share price data, processing it, and preparing it for comparison. The domain layer represents the core concepts of the system and remains independent from technical concerns. The infrastructure layer handles external operations such as accessing market data and storing cached data locally.

**Dependency Direction**

The system follows the principle that dependencies should flow inward. This means that outer layers depend on inner layers, but inner layers do not depend on outer layers. As a result, the core logic of the system remains independent from frameworks, APIs, and storage mechanisms. This makes the design more stable, as changes in external components do not directly affect the core of the application.

**Use of Interfaces**

Interfaces are used to separate the application logic from the infrastructure layer. The application layer defines the required behaviour through interfaces, while the infrastructure layer provides the concrete implementations. For example, data retrieval and caching are handled through interfaces, allowing different implementations to be used without changing the main logic of the system.

**Separation of Concerns**

The design ensures that responsibilities are clearly separated. Business logic is contained within the service layer and is not mixed with API calls or database operations. This helps to avoid tightly coupled code and makes the system easier to understand and maintain.

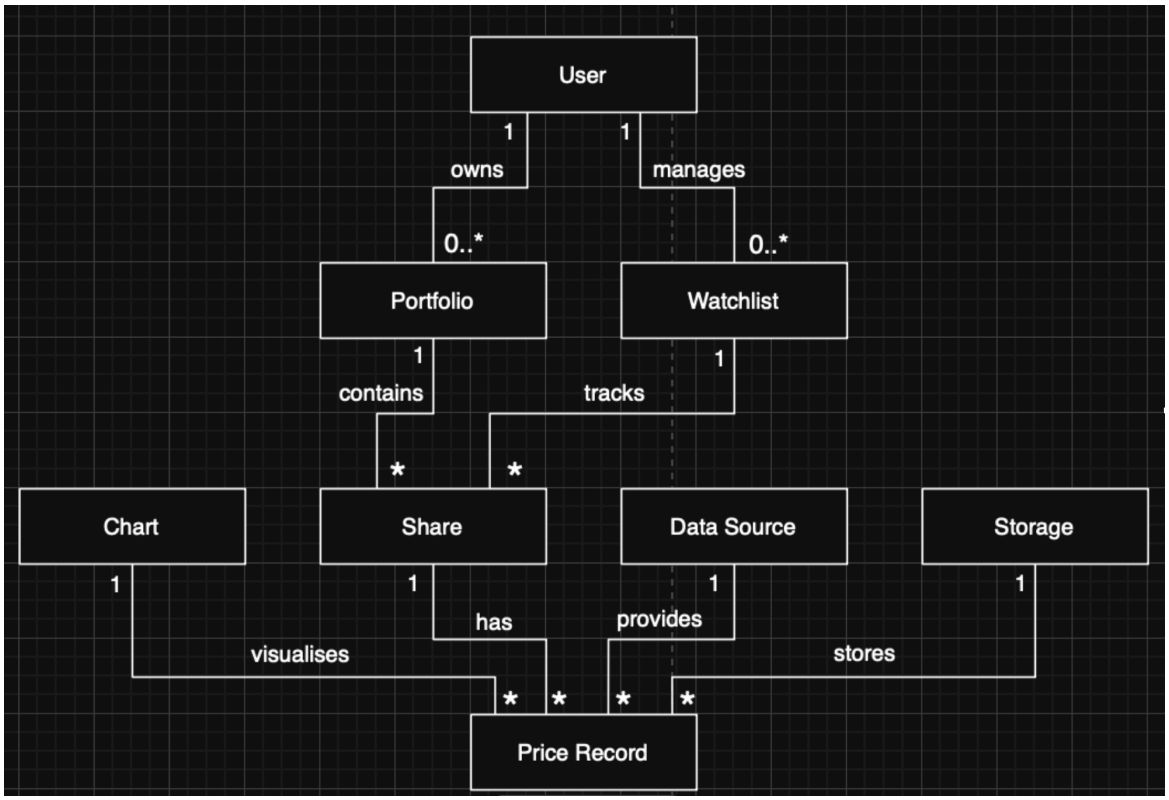
**Summary**

Overall, the system follows Clean Architecture principles by organising components into clear layers, controlling the direction of dependencies, and using interfaces to reduce coupling. This results in a design that is flexible, maintainable, and suitable for future development.

# Business Concept Model

**Description**

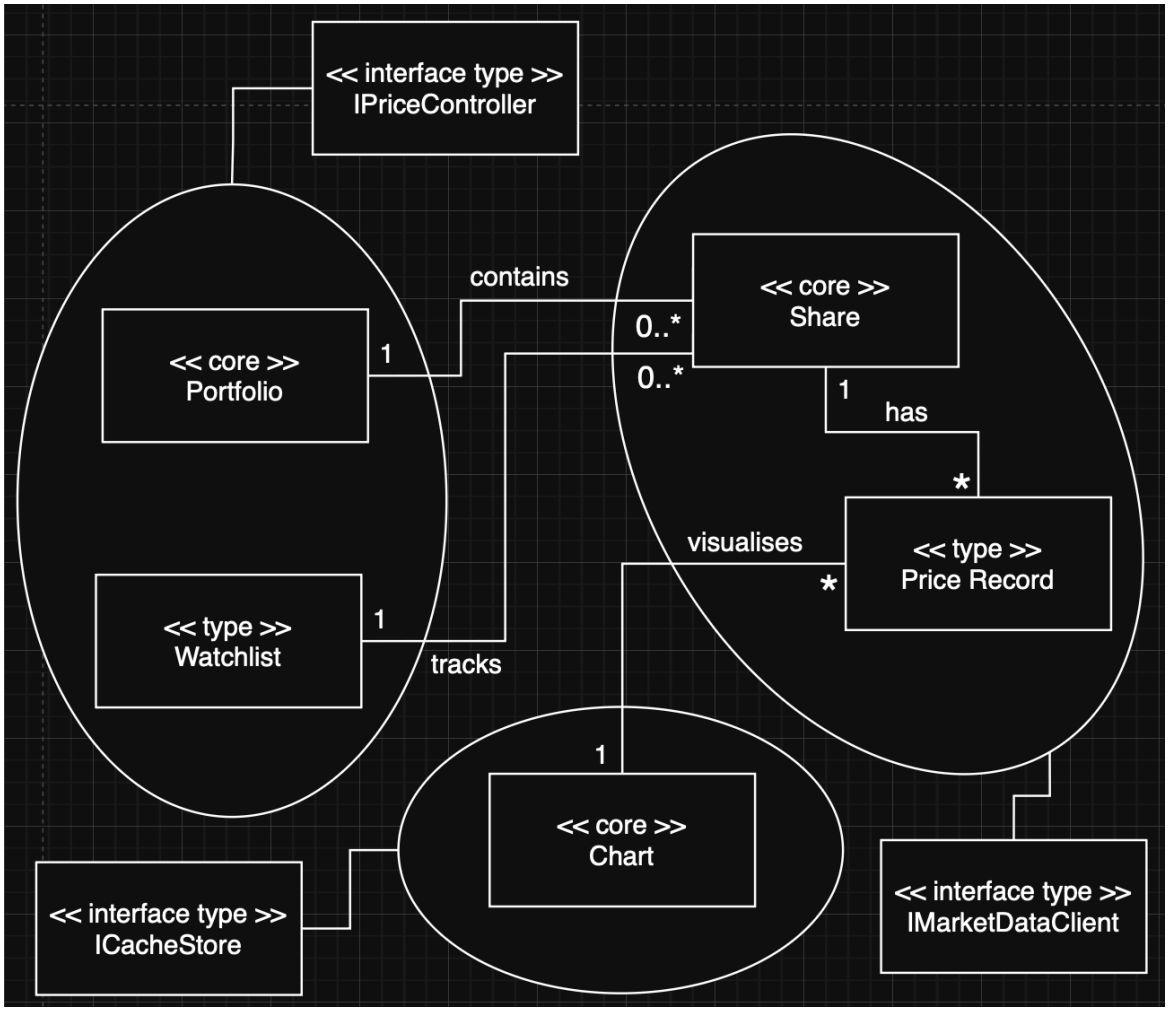
The business concept model shows the main real world parts of the Share Price Comparison system and how they are connected. It focuses on things that actually exist, such as users, shares, portfolios, and price records, rather than how the system is built. The user is the main part of the system and can manage portfolios and watchlists, which are used to organise shares. Each share has price records that show how its value changes over time. These records are used to create charts so the user can see trends. The data comes from an external source and can also be stored locally so it can still be used without internet. This model helps clearly define what the system is about before moving on to more detailed designs.



# Business Type Model

**Description**

The business type model takes the concept model and makes it more structured by defining types and how they are grouped. Each group represents a different part of the system and has one main core type that everything else in that group depends on. For example, the portfolio group is centred around the portfolio, while the share group is centred around the share. Other types, like price data and charts, support these core types. Interfaces are only used where needed to show how different groups connect, instead of adding too many unnecessary ones. This makes the model clearer and easier to understand while still showing how the system is organised.



# Component Specification Diagram

**Architectural Concepts and Their Application**

The architecture of the Share Price Comparison Web Application follows a layered architectural structure consisting of Presentation, Application, and Infrastructure layers. This structure is intentionally reflected in the component specification diagram, where each layer is clearly separated and responsibilities are distributed logically.

At the top of the diagram, the Presentation layer contains the Web UI and the PriceController. The Web UI represents the user interface through which stock symbols and date ranges are submitted. The PriceController acts as the entry point into the system, receiving HTTP requests and forwarding them to the Application layer. This separation ensures that user interaction logic remains independent from business processing logic.

The Application layer forms the core of the system. The PriceComparisonService coordinates the primary use case of the application, which includes retrieving price data and comparing up to two symbols. It enforces the business rules shown in the diagram, including the restriction that the date range must not exceed two years. It also implements the offline first behaviour by determining whether to retrieve data from the external provider or load it from the local cache. The ChartBuilder component, also within the Application layer, transforms retrieved price data into a structured dataset suitable for graphical representation. Separating chart preparation from price retrieval improves cohesion and keeps responsibilities focused.

The Infrastructure layer contains the MarketDataClient and the LocalCacheStore. The MarketDataClient connects to the External Market Data API using HTTPS, as shown in the diagram. The LocalCacheStore manages persistence through SQLite or JSON based storage, represented by the Local Storage component. By isolating these external interactions within the Infrastructure layer, the architecture reduces coupling between business logic and technical implementation details. If the data provider or storage mechanism changes, only this layer requires modification.

The direction of dependencies in the diagram demonstrates a controlled flow from Presentation to Application to Infrastructure. Higher layers depend on lower layers, but infrastructure components do not depend on application logic. This ensures maintainability, testability, and extensibility. Each component in the diagram has a clearly defined responsibility, reflecting high cohesion and structured separation of concerns.

Overall, the architecture directly mirrors the diagram structure and demonstrates a practical implementation of layered design principles.

**How the Architecture Supports Requirements and Project Goals**

The architecture shown in the component specification diagram directly supports the functional requirements and broader project objectives.

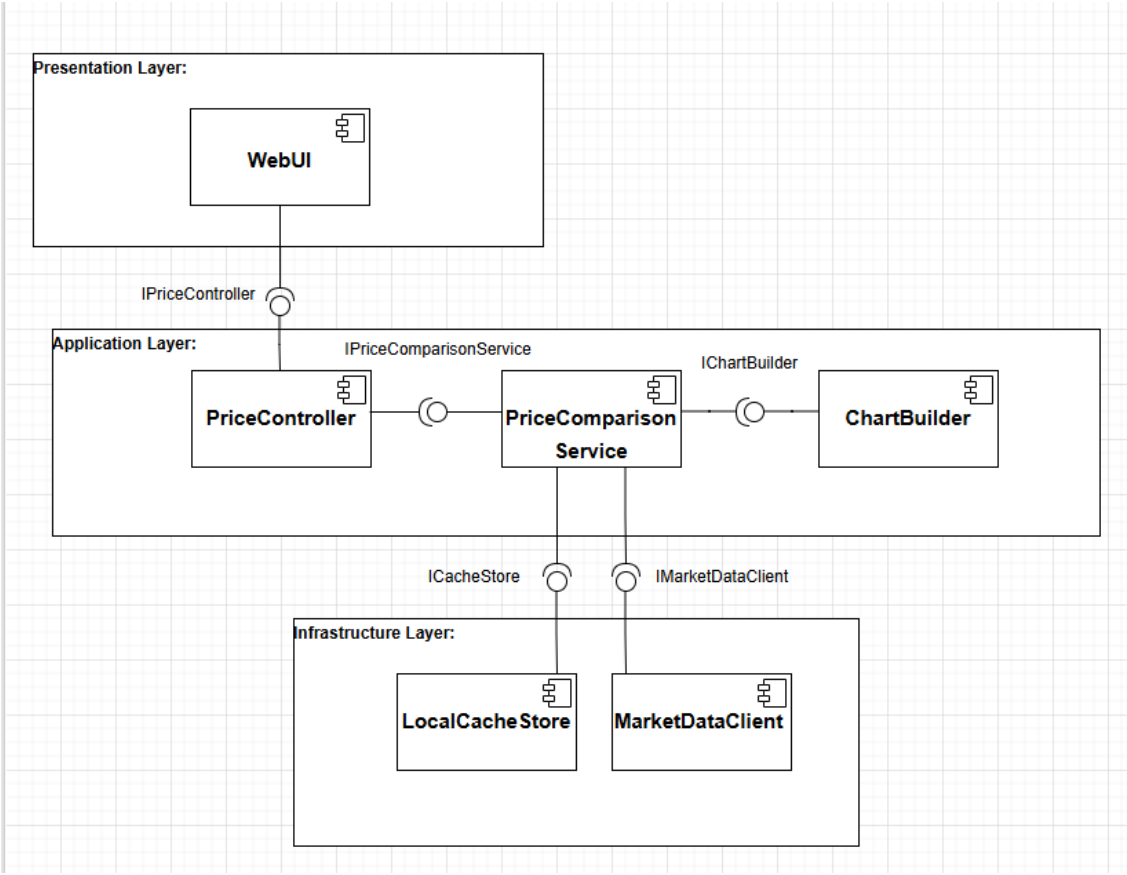
The requirement to retrieve daily share prices within a selected date range is implemented through the interaction between the PriceController and the PriceComparisonService. The service validates the date range to ensure it does not exceed two years, as indicated in the diagram. This ensures that constraints are enforced consistently at the business logic level.

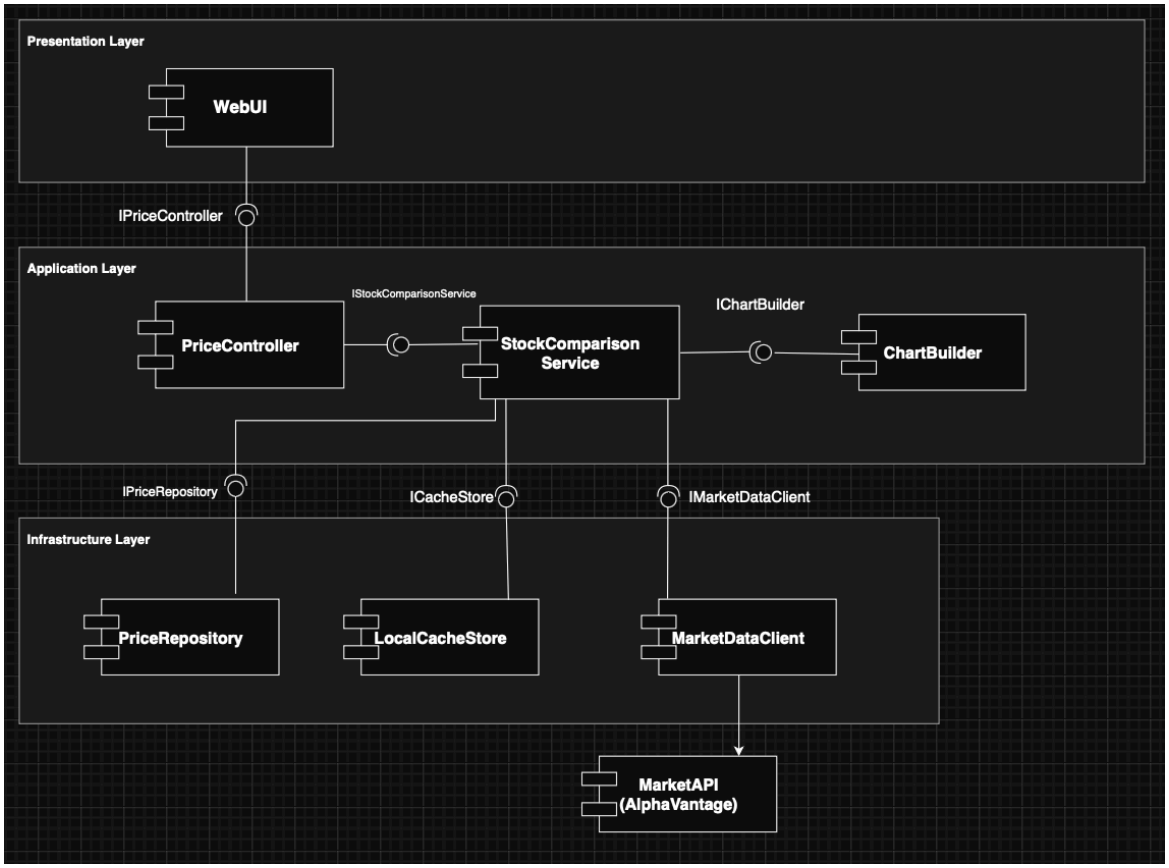
The ability to compare up to two symbols is handled within the PriceComparisonService. By centralising comparison logic within this component, the architecture ensures that all comparison related behaviour is managed in one location. This simplifies future expansion and improves reliability.

Offline functionality is achieved through the interaction between the PriceComparisonService, the MarketDataClient, and the LocalCacheStore. When the system is online, the service retrieves data from the external API through the MarketDataClient and then stores it locally using the LocalCacheStore. When offline, the service loads previously stored data from local storage. This workflow, illustrated in the diagram, ensures that the system remains partially usable without network connectivity.

Chart visualisation is supported through the ChartBuilder component. After price data is retrieved or loaded from cache, it is passed to ChartBuilder to produce a dataset suitable for rendering in the user interface. This separation keeps data processing distinct from presentation logic.

From a broader perspective, the architecture supports scalability and maintainability. The clear separation between layers ensures that changes to the user interface, business logic, or external integrations can be made independently. The diagram reflects a structured and well reasoned system design that aligns with both immediate requirements and long term project goals.


Presentation Layer:
WebUI
IPriceController
Application Layer:
IPriceComparisonService
IChartBuilder
PriceController
PriceComparison Service
ChartBuilder
ICacheStore
IMarketDataClient
Infrastructure Layer:
LocalCache Store
MarketDataClient


Presentation Layer
WebUI
IPriceController
Application Layer
IStockComparisonService
IChartBuilder
PriceController
StockComparison Service
ChartBuilder
IPriceRepository
ICacheStore
IMarketDataClient
Infrastructure Layer
PriceRepository
LocalCacheStore
MarketDataClient
MarketAPI (AlphaVantage)

# Dependency Structure

**Overview**

The dependency structure of the system was reviewed to ensure that components are organised correctly and that dependencies follow a clear and logical direction. The aim was to confirm that the system avoids tight coupling and supports a maintainable design.

**Dependency Relationships**

The system does not contain any circular dependencies between classes or packages. Each component depends only on what is necessary, and the overall flow of control is clear. The user interacts with the UI, which passes requests to the controller. The controller then calls the service layer, and the service layer interacts with external functionality through defined interfaces. This creates a clear and predictable flow through the system.

**Layer Interaction**

The infrastructure layer depends on interfaces defined in the application layer rather than the other way around. This ensures that the core logic of the system is not directly tied to specific implementations. As a result, changes to external systems such as APIs or storage mechanisms can be made without affecting the main application logic.

**Loose Coupling and Modularity**

The structure supports loose coupling by separating responsibilities across layers and using interfaces between them. This means that individual components can be modified or replaced without causing issues in other parts of the system. The package structure reflects this separation, with distinct layers for presentation, application, domain, and infrastructure.

**Testability and Scalability**

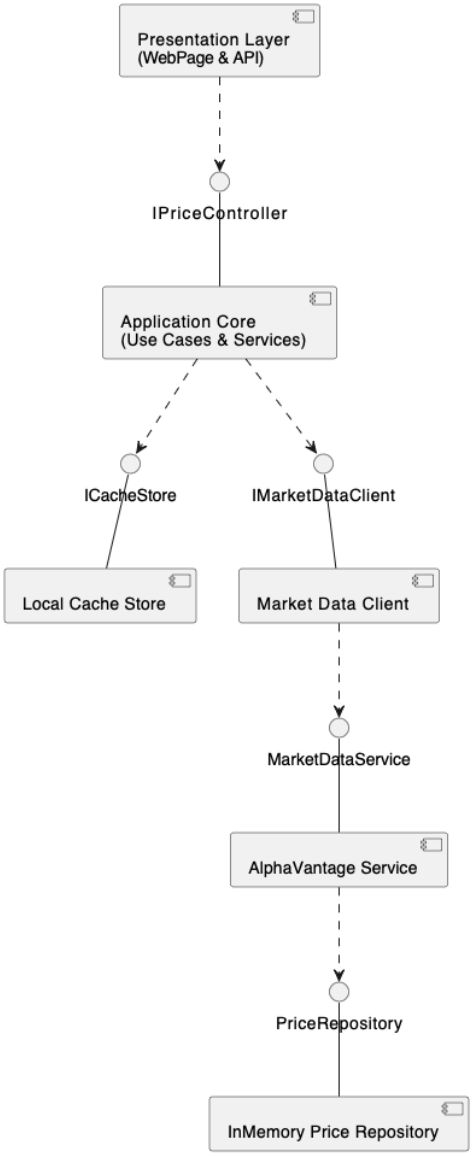
The dependency structure also improves testability, as components can be tested independently by using interfaces instead of concrete implementations. It also supports scalability, since new features or components can be added without disrupting the existing system. This makes the design more suitable for future development.

**Summary**

Overall, the dependency structure is clear, logical, and well organised. It avoids unnecessary dependencies, maintains loose coupling, and supports both testing and future growth of the system.

# Initial System Architecture Diagram

The architecture follows a clean, interface-driven design that separates the core business logic from technical details. The Application Core acts as the central brain, orchestrating the system by consuming multiple interfaces like ICacheStore and IMarketDataClient to decide where and how to fetch pricing data. Most of the lower-level components, such as the Local Cache Store, AlphaVantage Service, and InMemory Price Repository, are designed with single-interface implementations to ensure they only have one responsibility. This setup uses interfaces as protective barriers between layers, allowing you to swap out the external AlphaVantage API or the in-memory storage without needing to rewrite the core application rules, resulting in a system that is both easy to test and highly flexible for future upgrades.



# Use Case Model

**Overview**

The use case diagram represents how a user interacts with the Share Price Comparison App and outlines the main functionality provided by the system. It identifies the different actions available to the user and shows how the system supports these actions through internal processes such as data retrieval, caching, and storage.

**Actor**

The system has one primary actor, which is the user. The user interacts with the application to search for share data, view prices, and perform comparisons.

**Main Use Cases**

The diagram includes several key use cases that represent the core functionality of the system.

The user can search for a share symbol and select a date range to define the period of interest. The user can also select and convert currency, allowing share prices to be viewed in different formats depending on preference.

The system allows the user to view current share prices as well as historical data. This enables the user to analyse trends over time. In addition, the user can compare share prices between different companies and view the results in a chart format.

The user is also able to update share prices, ensuring that the data being viewed is accurate and up to date.

**Supporting System Use Cases**

Some use cases represent internal system behaviour rather than direct user interaction.

Retrieve Share Price Data is used by multiple user actions such as viewing prices, comparing shares, and updating data. This shows that many features rely on a common data retrieval process.

Store Price Data ensures that retrieved data is saved locally so that it can be accessed later. This supports offline functionality and improves performance.

Retrieve Cached Data is used when displaying charts, allowing the system to reuse previously stored data instead of always making external requests.

**Sequence of Interaction**

A typical interaction begins when the user searches for a share symbol and selects a date range. The system then retrieves the relevant share price data.

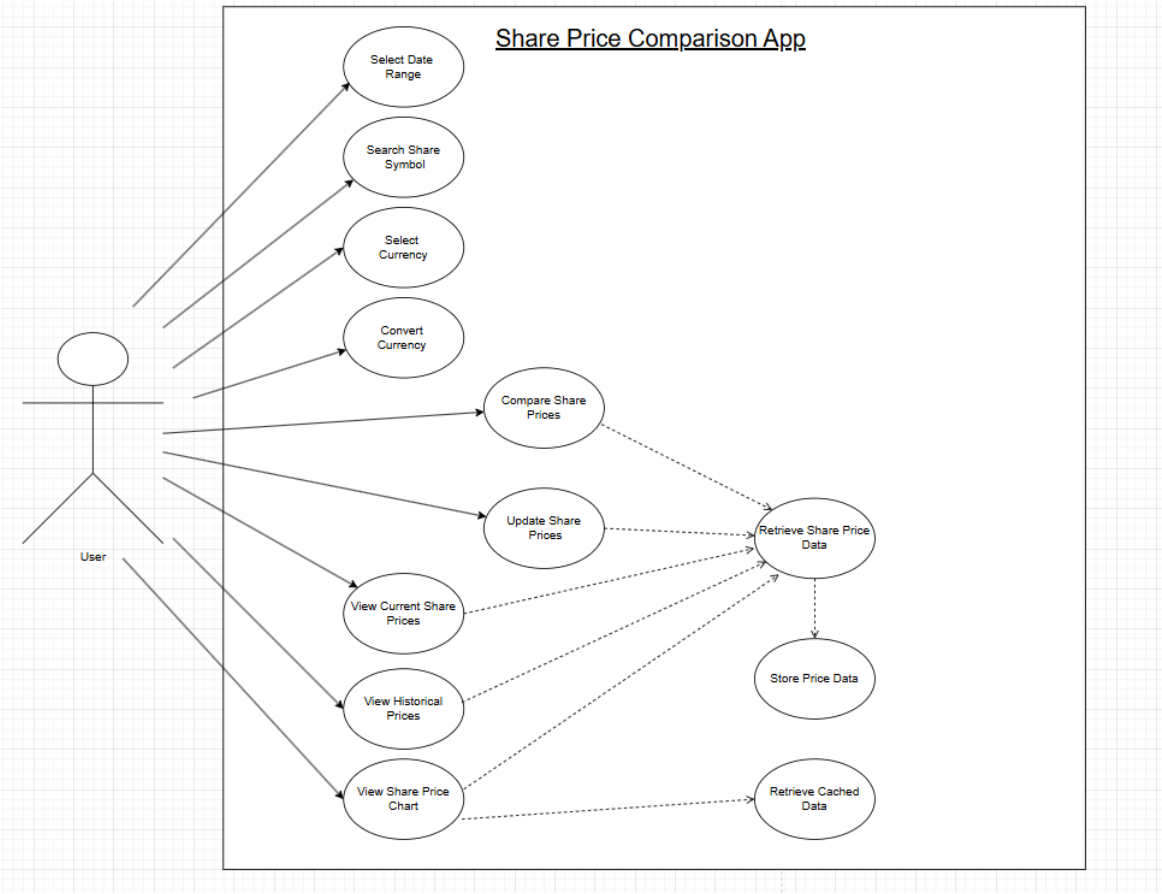
Once the data has been retrieved, it can be stored locally. The user can then choose to view current prices, view historical trends, or compare different shares.

If the user chooses to view a chart, the system may retrieve cached data to improve performance. The user can also update share prices at any time, which triggers another data retrieval process.

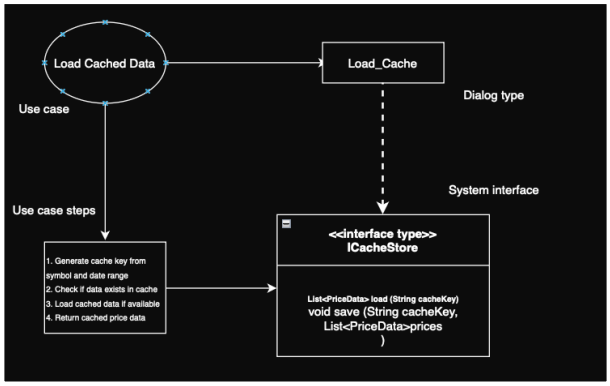
Currency selection and conversion can be applied at different stages, allowing the displayed data to be adapted to the user's preference.

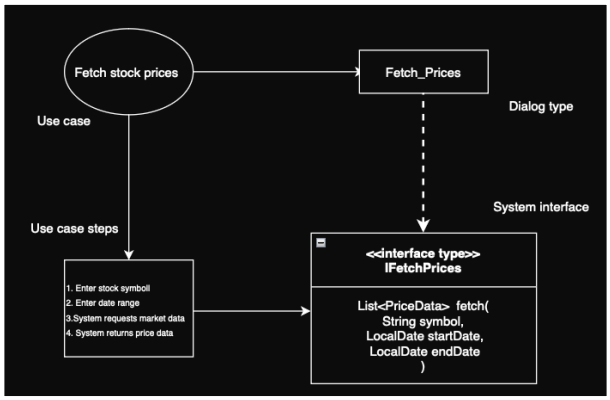
**Summary**

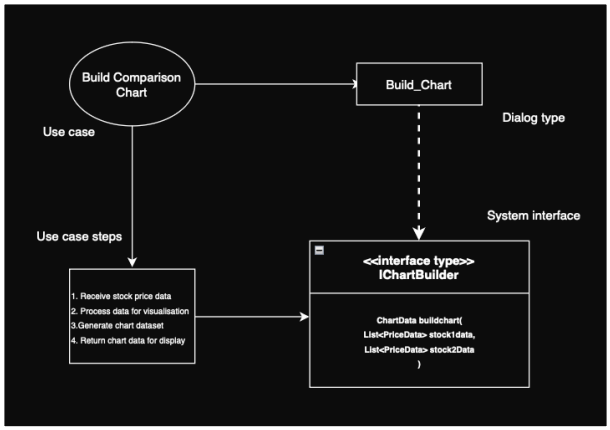
Overall, the use case diagram demonstrates the main capabilities of the system and how user actions are supported by internal processes. It shows a clear separation between user-driven functionality and system-level operations, while also highlighting how different features depend on shared data retrieval and storage mechanisms.

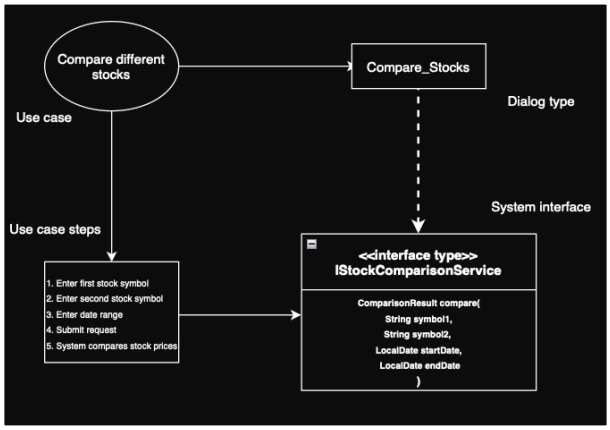


# Interface Diagrams



# Compound Component Diagram

## Description

This diagram shows the composite structure of the Stock Comparison System. It illustrates how external clients interact with the system through defined interfaces, how the application layer coordinates core use cases, and how the infrastructure layer handles data retrieval and storage. The diagram also highlights the introduction of a pipeline pattern, where stock price data is processed through a series of filters before being used in comparisons and visualisation.

**System Overview**

- The system is encapsulated within BigStockComparisonSys
- External clients access it through a single interface
- Internal complexity is hidden behind ports and interfaces

**Application Layer**

- PriceComparisonService acts as the central controller
- It delegates tasks to:
  - FetchSharePricesUseCase
  - CompareSharesUseCase
  - ChartBuilder
- This keeps logic modular and easy to maintain

**Pipeline Addition**

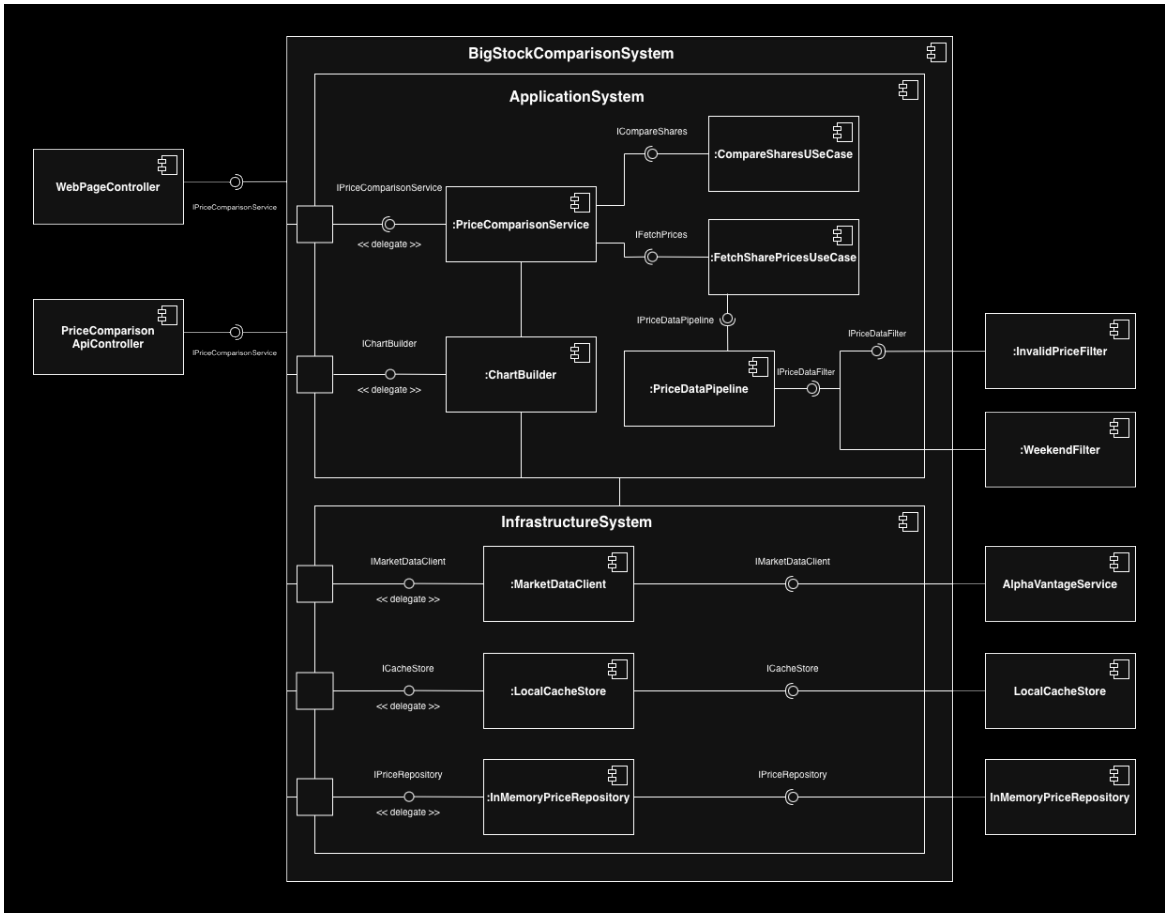
- FetchSharePricesUseCase now uses PriceDataPipeline
- The pipeline processes raw data before it is used
- Filters include:
  - InvalidPriceFilter
  - WeekendFilter
- Each filter implements a shared interface
- This allows filters to be added or removed easily

**Infrastructure Layer**

- Handles all external communication and storage
- MarketDataClient retrieves data from AlphaVantageService
- LocalCacheStore reduces repeated API calls
- InMemoryPriceRepository stores processed data

**Connections**

- Lollipop represents a provided interface
- Socket represents a required interface
- Assembly connectors link components together
- Delegation connectors pass requests through ports


BigStockComparisonSystem
ApplicationSystem
WebPageController
PriceComparison ApiController
IPriceComparisonService
IPriceComparisonService
IPriceComparisonService
<< delegate >>
:PriceComparisonService
ICompareShares
:CompareSharesUSeCase
IFetchPrices
:FetchSharePricesUseCase
IChartBuilder
<< delegate >>
:ChartBuilder
IPriceDataPipeline
:PriceDataPipeline
IPriceDataFilter
IPriceDataFilter
:InvalidPriceFilter
:WeekendFilter
InfrastructureSystem
IMarketDataClient
<< delegate >>
:MarketDataClient
IMarketDataClient
AlphaVantageService
ICacheStore
<< delegate >>
:LocalCacheStore
ICacheStore
LocalCacheStore
IPriceRepository
<< delegate >>
:InMemoryPriceRepository
IPriceRepository
InMemoryPriceRepository

# Architectural Patterns

## Domain Independence Styles

The app uses specific styles to keep different parts of the code separate. This ensures that the core business logic (what the app actually does) isn't messy or tied too closely to the user interface or external databases.

**Layered and N-Tiered Architecture**

The project follows a layered structure, which basically means the code is organised into "floors." Each layer has a specific job and can only talk to the layer directly below it.

Presentation Layer: This is the "front door" where HTTP requests come in. It handles routing and turns user input into something the app can understand.

Application Layer: This is like the middleman or orchestrator. It contains the "use cases," like fetching share prices or comparing them.

Domain Layer: This is the most important part. It holds the core data models (like PriceData). It’s "pure," meaning it doesn't rely on any other part of the system to work.

Infrastructure Layer: This handles the "outside world," such as connecting to the AlphaVantage API or saving data to a repository.

**Model-View-Controller**

To manage the web side of things, we used the MVC pattern through the Spring Framework.
The Model holds the data (like comparison results).
The View (Thymeleaf templates) is the actual webpage the user sees.
The Controller (WebPageController.java) handles the logic, like taking a URL request and telling the application layer what to do with it.

**Pipes and Filters**

For processing data, the system uses a Pipes and Filters style. Think of it like an assembly line.
The Pipe (PriceDataPipeline.java) is the belt that moves the data along.
The Filters (like WeekendFilter.java) are stations that clean the data. For example, removing stock prices from the weekend when markets are closed. This makes it really easy to add new cleaning steps later without breaking the whole process.

**Adapter Pattern**

We used the Adapter Pattern to make sure our app isn't too dependent on third-party services. Instead of connecting the app directly to AlphaVantage, we created a "bridge" (MarketDataClient.java). If we ever want to switch to a different provider like Yahoo Finance, we just have to write a new adapter rather than rewriting the whole app.

## Service-Oriented Architecture (SOA) Principles

Even though this is a single application, it’s designed using SOA principles. This means the different parts of the app act like independent services that communicate with each other.

**Service Modularity**

Instead of having one giant class that does everything (a "god class"), we broke the code down into small, focused services using interfaces. For example:
IFetchPrices only handles getting data.
ICompareShares only handles the math for comparing stocks.
This modularity means you can change the math logic without accidentally breaking the data-fetching logic.

**Interoperability**

Interoperability is about how well different parts of a system work together. In StockScope, we use interfaces as "contracts."
Components don't care how a task is done; they just care that it gets done according to the interface. For example, the WebPageController talks to an interface (IPriceController) rather than a specific piece of code. This makes the system very flexible - it’s almost like a group of mini-apps working together, which would make it very easy to turn this into a full micro-services system in the future.

# Meetings

Meeting Report - 12/02/2026 (15:30–16:30)

**Project**

Stock Comparison Application

**Meeting Goal**

Confirm project brief, sprint deadlines/marks

Agree app scope (features in/out)

Set up workflow (Kanban + GitHub)

Assign Sprint 1 tasks (requirements, architecture, diagram and code layout)

**Roles**

Facilitator: Luke Pring

Note taker: Sameer Shabbir

**Attendees**

Luke Pring, Jack Turner, Azlan Rashid, Sameer Shabbir

**Roundtable Updates**

Luke: Confirmed project scope + Sprint 1 focus.

Jack: Will help with repository setup and governance documentation.

Azlan: Will research architecture + start component diagram.

Sameer: Will write meeting reports and help set up repo/board.

**Discussion points**

Went through coursework brief, sprint deadlines, and deliverables

Agreed core features: fetch prices, store data, show graphs, compare 2 shares

Decided to use GitHub + Kanban and follow sprint workflow

Split Sprint 1 tasks (requirements, architecture, diagram, Java skeleton, meeting notes)

**Actions**

Set up GitHub repo + Kanban Board (Luke + Sameer)

Contribute to Code of Conduct + PR rules (Jack + Luke)

Start architecture research + component diagram (Azlan)

Write meeting reports + draft requirements (Sameer)

Meeting Report - 16/02/2026 (13:00–13:45)

**Project**

Stock Comaprison Application

**Meeting Goal**

Review Sprint 1 deliverables against the mark scheme

Demo the Java project builds and match architecture/diagram

Collect issues/feedback and convert them into Sprint 2 tasks

**Roles**

Facilitator: Luke Pring

Note taker: Sameer Shabbir

**Attendees**

Luke Pring, Jack Turner, Azlan Rashid, Sameer Shabbir

**Roundtable Updates**

Luke: Checked Sprint 1 work matches the marking criteria.

Jack: Reviewed repository governance and contribution guidelines.

Azlan: Presented the architecture diagram and explained it.

Sameer: Completed Sprint 1 meeting reports and organised evidence for submission.

**Discussion points**

Reviewed Sprint 1 deliverables (requirements, architecture, diagram, code)

Checked repo evidence: branches, commits, project board, Code of Conduct

Noted improvements needed for Sprint 2 (real API fetch, storage)

Created Sprint 2 backlog tasks from feedback

**Actions**

Review Sprint 1 for code review + fix any issues (Luke + Sameer)

Support team with repository processes (Jack + Azlan)

Final check of architecture diagram to ensure it matches code structure (Azlan)

Update Kanban with Sprint 2 tasks from feedback (Jack)

Meeting Report - 18/02/2026 (13:00–14:00)

**Project**

Stock Comparison Application

**Meeting Goal**

Plan Sprint 2 implementation tasks

Split responsibilities (backend/frontend)

Update Kanban board with Sprint 2 backlog + priorities

**Roles**

Facilitator: Luke Pring

Note taker: Sameer Shabbir

**Attendees**

Luke Pring, Jack Turner, Azlan Rashid, Sameer Shabbir

**Roundtable Updates**

Luke: Planned Sprint 2 priorities and task split.

Jack: Will contribute to frontend/UI implementation.

Azlan: Updated the architecture plan for Sprint 2 features.

Sameer: Continued meeting reports and tracked tasks/updates on the board.

**Discussion points**

Planned Sprint 2 build tasks and priorities

Discussed offline support using stored/cached data

Confirmed approach for retrieving share prices

Assigned backend vs frontend responsibilities and updated Kanban board

**Actions**

Add Sprint 2 tasks + priorities to the Kanban board (Luke + Jack)

Start Sprint 2 coding (Sameer + Jack)

Update the architecture diagram for Sprint 2 (Azlan)

Support team with implementation tasks (All team members)

Meeting Report - 06/03/2026 (13:00–14:00)

**Project**

Stock Comparison Application

**Meeting Goal**

Finalise system architecture

Make sure it follows Clean Architecture

Review business type model

**Roles**

Facilitator: Luke Pring

Note taker: Sameer Shabbir

**Attendees**

Luke Pring, Jack Turner, Azlan Rashid, Sameer Shabbir

**Roundtable Updates**

Luke: Advised to base the business type model on the business concept model

Jack: Worked on UI and reviewed system structure

Azlan: Finalised architecture diagram

Sameer: Updated meeting notes and GitHub

**Discussion points**

Confirmed layered architecture (presentation, application, infrastructure)

Agreed to use interfaces for better structure

Reviewed system flow (UI → Controller → Service → Data)

Confirmed use of API + cache for data

**Actions**

Upload architecture diagram (Sameer)

Complete business type model (Azlan)

Continue UI work (Jack)

Review and confirm architecture design (Luke)

Update GitHub (All)

**Outcome**

Architecture completed and uploaded. System follows a clear layered structure and meets Sprint 2 requirements.

Meeting Report - 13/03/2026 (13:00–14:00)

**Project**

Stock Comparison Application

**Meeting Goal**

Start Sprint 2 planning

Discuss architecture approach

Assign initial tasks

**Roles**

Facilitator: Luke Pring

Note taker: Sameer Shabbir

**Attendees**

Luke Pring, Jack Turner, Azlan Rashid, Sameer Shabbir

**Roundtable Updates**

Luke: Outlined Sprint 2 requirements and tasks

Jack: Planned UI development

Azlan: Began working on architecture design

Sameer: Updated meeting notes and project board

**Discussion points**

Reviewed Sprint 2 requirements

Discussed layered architecture (presentation, application, infrastructure)

Planned use of API and local cache

Decided to split work between frontend and backend

**Actions**

Begin architecture diagram (Azlan)

Start backend setup (Sameer)

Start UI design (Jack)

Monitor progress and support team (Luke)

Update GitHub and Kanban board (All)

**Outcome**

Sprint 2 started with clear tasks and responsibilities. Initial architecture planning began.

Meeting Report - 27/03/2026 (15:00–16:00)

**Project**

Stock Comparison Application

**Meeting Goal**

Present Sprint 2 to Lecturer

Get feedback for Sprint 3

**Roles**

Facilitator: Luke Pring

Note taker: Jack Turner

**Attendees**

Luke Pring, Jack Turner, Azlan Rashid, Sameer Shabbir

**Discussion points**

Lecturer reviewed our sprint 2 progress and failed us.

Update business concept model - Comparison isn't an entity, change it to comparison result not comparison functionality, User needs to have portfolio and watch list, each group needs a <<core>>, not all the groups need an interface.

For use case diagram user must be able to update share prices, view and use different currencies etc

Initial system architecture diagram needs to be created.

Component specification diagram.

**Actions**

Redo use case diagram (Azlan)

Redo component specification diagram (Sameer)

Redo Business concept model and update type model (Jack)

Create initial system architecture diagram (Luke)

Update GitHub and Kanban board (All)

**Outcome**

Sprint 2 has been failed, however we have been given another chance to present over the Easter break, so we will correct everything that the lecturer mentioned was wrong and present it again.

Meeting Report - 09/04/2026 (10:00–10:30)

**Project**

Stock Comparison Application

**Meeting Goal**

Present Sprint 2 to Lecturer again

**Roles**

Facilitator: Luke Pring

Note taker: Jack Turner

**Attendees**

Luke Pring, Jack Turner, Azlan Rashid, Sameer Shabbir

**Discussion points**

Lecturer reviewed our sprint 2 progress and passed us.

Add interface diagram (see week 4 lab solutions)

Add back the interfaces to the type model

Match the interface names of the type model to the initial system architecture

**Actions**

Add interface diagrams (Sameer)

Update type model (Jack)

Update GitHub and Kanban board (All)

**Outcome**

Sprint 2 has now been passed, we will add all the items discussed to our project ready for sprint 3.

Meeting Report - 10/04/2026 (12:00–13:00)

**Project**

Stock Comparison Application

**Meeting Goal**

Work out what needs to be completed for Sprint 3

Assign tasks to each of us

**Roles**

Facilitator: Luke Pring

Note taker: Jack Turner

**Attendees**

Luke Pring, Jack Turner, Azlan Rashid, Sameer Shabbir

**Discussion points**

Read through the coursework specification and our notes from in class to determine what needs to be completed for Sprint 3. The following is what needs to be done:

- Identify Compound components and create a diagram.
- Identify any domain independent styles that we used and add any that could improve code.
- Identify whether we have used any principles of SOA, add if not.
- Create test cases and document results.
- Create project report.
- Create powerpoint for presentation.

**Actions**

- Create test cases and document results (Azlan)
- Identify compound components and create a diagram, identify domain independent styles, identify SOA principles, create project report (including: all architectural designs form GitHub, compound component diagram, architectural patterns, team meetings, test cases) (Jack)
- Identify compound components, identify domain independent styles and add 'pipes and filters', identify SOA principles, create slides for presentation (Luke)
- Update GitHub and Kanban board (All)

**Outcome**

Sprint 3 has been planned well and will be able to be completed to a standard that is acceptable to the lecturer.

# Test Cases

| No | Objective | Input | Expected Result | Actual Result | P/F | Notes |
|---|---|---|---|---|---|---|
| **1** | Check the main functionality of the system by comparing two valid shares within an acceptable date range. | AAPL and MSFT with 5 month range | The system should retrieve historical data for both shares and display them on a single graph with correct labels. | Data is successfully retrieved and both shares are displayed clearly on the graph. | Pass | Range was selected as this is the limit for the free API key we used, however the system is capable of accepting longer ranges. |
| **2** | Blank field verification - this test checks that the system enforces required input fields and prevents incomplete submissions. | Leave one ticket field empty and attempt to compare. | System should display a missing field message and not process the request. | The system prevents submission and shows a validation message. | Pass | N/A |
| **3** | Invalid ticker symbol - this checks how the system handles incorrect or non existent ticker symbols. | XXXX and APPL with a valid date range of 5 months | System should show feedback that the ticker is invalid and not attempt to display the incorrect data. | System displays an error message and does not crash. | Pass | N/A |
| **4** | Offline Data Retrieval - checks the system's ability to use stored data when the external data source (AlphaVantage) is unavailable. | Disconnect network after previously retrieving share data. | The system loads stored data instead of failing | Previously stored data is displayed successfully. | Pass | Discussed with group ways to make it clearer that cached data was used as the network wasn't available. |

| | | | | | | |
|---|---|---|---|---|---|---|
| **5** | Date range impact test - checks that changing the data range correctly affects the displayed data. | Compare the same shares using different data ranges. AAPL, MSFT, date range of 1 year. | The graph updates to reflect the selected time period with different data values. | Input of 1 year ignored - only 5 months of data shown on the graph and table. | Fail | After assessing within the group, we found that the free API key we were using limited data to 5 months, so while nothing was wrong with our code, we were limited to this range. **This was fixed** by updating axis labels and by adding clearer error messaging. |
| **6** | Invalid Date Range - Checks if the system validates that the start date comes before the end date. | Start Date: 01-01-2025 End Date: 01-01-2026 | System should display a validation error message stating that the start date must precede the end date, preventing submission. | The system correctly identifies the invalid range and shows an error message without processing the request. | Pass | N/A |
| **7** | Case Insensitivity Test - Checks that user input for ticker symbols is not case-sensitive. | aapl and msft (lowercase) with a valid 5-month date range. | System should automatically format the lowercase input to uppercase and successfully retrieve and display the data. | Data is successfully retrieved, the lowercase input is handled gracefully and displayed correctly on the graph. | Pass | N/A |
| **8** | Identical ticker comparison - Checks system behaviour when a user compares a stock against itself. | TSLA and TSLA with a valid 5-month date range. | System should either process the request and display perfectly overlapping lines, or inform the user that identical tickers were used. | Data is retrieved; graph successfully displays the data with identical overlapping lines. | Pass | N/A |
| **9** | Future Date Validation - Tests how the system handles date ranges that extend into the future. | Start Date: 1 month ago End Date: 1 year in the future. | System should either fetch data up to the current actual date and ignore the future dates, or throw a validation error. | System ignores future dates, returning and plotting data up to the most recent available trading day. | Pass | N/A |

| | | | | | | |
|---|---|---|---|---|---|---|
| **10** | API Rate Limit Handling - Checks the system's resilience when the free API tier limits are exceeded. | Rapidly submit 6 comparison requests within a 1-minute window. | The system should gracefully handle the 429 Too Many Requests error and display a user-friendly message, rather than crashing. | System catches the API limit error and displays a clear message asking the user to wait a minute before trying again. | Pass | Crucial for free-tier AlphaVantage usage to ensure the application remains stable under heavy use. |